

**DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF TECHNOLOGY
DHARMSINH DESAI UNIVERSITY, NADIAD
B. TECH. CE SEMESTER – V
SUBJECT: (23CE520) ADVANCED TECHNOLOGIES**

Project Report on
**EQUITRACK – REAL-TIME INDIAN STOCK MARKET ANALYTICS &
TRACKING PLATFORM**

Submitted By:

1. Vaghasiya Prince

Roll No.: CE163

Student ID: 24CEUOS163

Email: 24ceuos163@ddu.ac.in

2. Vaghasia Frenil

Roll No.: CE162

Student ID: 24CEUOS162

Email: 24ceuos162@ddu.ac.in

Guided By:

Prof. Vrund Dobariya

Assistant Professor

Department of Computer Engineering

Faculty of Technology

Dharmsinh Desai University, Nadiad

Academic Term: 2026–2027

Date of Submission: October 2026

CERTIFICATE

This is to certify that the project entitled "**EquiTrack – Real-Time Indian Stock Market Analytics & Tracking Platform**" is a bonafide work carried out by:

Vaghasiya Prince (Roll No: CE163, Student ID: 24CEUOS163)

Vaghasia Frenil (Roll No: CE162, Student ID: 24CEUOS162)

students of **B. Tech. Semester – V, Computer Engineering**, during the academic year 2026–2027 in partial fulfillment of the requirements for the subject **(23CE520) Advanced Technologies** at **Faculty of Technology, Dharmsinh Desai University, Nadiad**.

The work presented in this report is original and has been carried out under the guidance of the faculty mentor and approved by the department.

Prof. Vrund Dobariya
Project Guide
Assistant Professor, CE
Dept.

Head of Department
Department of Computer
Engineering
Faculty of Technology,
DDU

External Examiner
Faculty of Technology
Dharmsinh Desai
University

Date: 08 / 10 / 2026

Place: Nadiad

TABLE OF CONTENTS

Certificate	2
Table of Contents	3
Abstract	4
1. Introduction	5
1.1 Brief Introduction & Project Motivation	5
1.2 Technology / Platform / Tools Used	6
2. Functional and Non-functional Requirements	7
2.1 Functional Requirements (SRS Alignment)	7
2.2 Non-functional Requirements	8
2.3 Design Constraints & Assumptions	9
3. UML Diagrams and Database Design	10
3.1 UML Use Case Diagram	10
3.2 System Architecture Diagram	11
3.3 UML Sequence Diagram	12
3.4 Database Schema & ER Design	13
4. Implementation Detail	15
4.1 Modules Created and Description	15
4.2 Core Algorithms, Flowcharts & Examples	16
5. Testing	18
5.1 Testing Framework & Methodology	18
5.2 Automated Test Cases Matrix (17 Test Cases)	19
6. Screen-shots	21
6.1 User Authentication Module	21
6.2 Live Market Dashboard	22
6.3 Interactive Multi-Timeframe Stock Charts	23
6.4 Watchlist & Market Pulse	24
7. Conclusion	25
8. Limitation and Future Extension	26
9. Bibliography	27

ABSTRACT

In modern financial technology, retail participation in the Indian equity markets (National Stock Exchange - NSE and Bombay Stock Exchange - BSE) has witnessed exponential growth. However, most existing financial platforms suffer from cluttered user interfaces, sluggish chart rendering, invasive advertising, and paywalled market data. **EquiTrack** is designed and implemented as an enterprise-grade, clean, responsive, and real-time stock market tracking and analysis web application adhering to modern full-stack web standards and the Apple Design Language.

The system is developed on the **MERN Stack** (MongoDB, Express.js, React 19, Node.js v22). It interfaces directly with the **Angel One SmartAPI** broker infrastructure using automated Time-based One-Time Password (TOTP) multi-factor authentication to ingest live equity quotes, market indices (NIFTY 50, SENSEX), and multi-timeframe historical candlestick feeds (1-minute, 5-minute, 15-minute, and 1-day candles). To tackle aggressive third-party API rate limits and avoid HTTP 403 Forbidden errors, EquiTrack introduces a dedicated in-memory instrument caching dictionary and a 30-second TTL batch-quote aggregation pipeline, reducing scrip resolution latency from over 1.2 seconds to under 10 milliseconds.

On the presentation layer, the application utilizes React 19 bundled via Vite and incorporates TradingView's high-performance **lightweight-charts** engine to provide 60fps pan, zoom, and crosshair interactions across multiple user-selectable intervals (5m, 1d, 1w, 1m). Security is enforced using JSON Web Tokens (JWT) for stateless bearer authorization alongside salted bcryptjs password hashing. An automated test suite developed with Node.js Native Test Runner (`node:test`) and Supertest validates all endpoints with a 100% pass rate. EquiTrack offers retail investors a seamless, high-speed, and aesthetically superior tracking experience without financial overhead.

1. Introduction

1.1 Brief Introduction & Project Motivation

The Indian equity ecosystem is among the most dynamic financial markets in the world, with tens of millions of active retail trading accounts registered across depository participants. Real-time market awareness, instant technical chart visualization, and systematic stock monitoring are vital requirements for making informed financial decisions. Despite the availability of numerous brokerage platforms, users routinely encounter critical shortcomings:

- **Cluttered and Distracting Interfaces:** Mainstream financial portals overwhelm users with aggressive banners, advertisements, unverified tips, and visually complex layouts that degrade user focus.
- **Latency and Sluggish Rendering:** Many legacy platforms render technical charts using heavy server-side scripts or bloated canvas implementations that suffer from frame drops during rapid navigation.
- **Fragmented Workflows:** Users frequently need to switch across multiple distinct tools to track major market indices, analyze company fundamentals, inspect multi-timeframe price charts, and maintain personalized watchlists.

EquiTrack was conceived to solve these challenges by providing a unified, high-performance, and distraction-free market analysis workspace. Styled in strict accordance with the **Apple Design System**, EquiTrack features a signature two-row navigation bar, Action Blue interactive accents, SF Pro typography with precise negative letter-spacing, and subtle micro-interactions. The platform delivers instant debounced scrip search, real-time market pulse monitoring, personalized watchlists, and interactive TradingView price charts with timeframe options tailored for Indian investors.

1.2 Technology / Platform / Tools Used

EquiTrack is constructed utilizing industry-standard, production-proven modern web technologies:

Layer / Category	Technology / Tool	Purpose & Justification
Frontend Framework	React 19 (JavaScript SPA)	Declarative component-driven UI with efficient virtual DOM reconciliation and reactive state management.
Build Tool & Bundler	Vite 8.3	Ultra-fast Hot Module Replacement (HMR) and optimized ES module

Layer / Category	Technology / Tool	Purpose & Justification
		bundling with an integrated backend API proxy.
Styling & Design System	Vanilla CSS (Apple Design System)	High-performance CSS with zero runtime overhead; precise design tokens, spring easing transitions, and dark/light modes.
Interactive Charting	TradingView Lightweight Charts (v5.2)	Canvas-rendered 60fps financial chart library providing Candlestick, Line, and Area spline charting with live crosshairs.
Backend Runtime & Server	Node.js v22 & Express.js v5.2	Asynchronous, non-blocking REST API service supporting asynchronous controller pipelines and route middleware.
Database & ODM	MongoDB & Mongoose ODM (v9.1)	Document-oriented NoSQL database providing flexible JSON schemas, compound indexes, and pre-save validation hooks.
Authentication & Security	JWT (jsonwebtoken) & bcryptjs	Stateless HTTP Bearer token authorization and one-way salted password hashing (10 salt rounds).
External Market Gateway	Angel One SmartAPI & otplib	Live market quotes, market breadth, and historical candle data ingested using automated TOTP authentication.
Testing Frameworks	Node.js Native Test Runner & Supertest	Automated end-to-end integration testing and HTTP endpoint assertions without external test-runner overhead.
Development Tools	VS Code, Postman, Git, GitHub	Code editing, REST API contract validation, version control, and collaborative repository management.

2. Functional and Non-functional Requirements

The system requirements for EquiTrack were formalized during Lab 02 (Software Requirements Specification) and refined during Lab 03 (REST API Specification). They are organized into discrete, testable functional modules alongside stringent non-functional performance and security parameters.

2.1 Functional Requirements

Module R.1: User Management

- **R.1.1 User Registration:** The system shall provide an endpoint (POST /api/auth/register) enabling new users to create accounts by submitting Full Name, valid Email Address, and Password. The system validates email uniqueness, enforces password length constraints (≥ 6 characters), and encrypts passwords using bcrypt before persistence.
- **R.1.2 User Login:** The system shall provide an authentication endpoint (POST /api/auth/login) that verifies email and password against stored cryptographic hashes. Upon successful authentication, the system returns a signed JWT token valid for 7 days.
- **R.1.3 View Profile:** The system shall provide an authenticated endpoint (GET /api/auth/profile) that retrieves the logged-in user's name, registered email address, and account creation timestamp based on the supplied JWT bearer token.
- **R.1.4 Update Profile:** The system shall provide an endpoint (PUT /api/auth/profile) permitting users to update their display name, email address (with duplicate collision checks), and modify their password securely.
- **R.1.5 User Logout:** The client application shall clear the stored JWT token and session context, revoking local authorization and redirecting the user to the Sign In interface.

Module R.2: Market Dashboard

- **R.2.1 Live Market Pulse:** The dashboard shall present high-level benchmark indices (NIFTY 50 and SENSEX) displaying current values, day points change, percentage change, and visual market trend status.
- **R.2.2 Top Gainers and Top Losers:** The system shall aggregate live equity performance across active NSE scrips, sorting equities into dedicated lists of the top 5 highest advancing equities and top 5 declining equities with dynamic price and percentage badges.
- **R.2.3 Suggested Stocks:** The dashboard shall showcase a curated 3-column store-style grid of prominent liquid Indian equities (e.g., HDFC Bank, Titan, Wipro, Asian Paints), enabling users

to click any card to inspect full company fundamentals.

Module R.3: Stock Management & Interactive Charts

- **R.3.1 Instant Debounced Stock Search:** The system shall provide an endpoint (GET /api/market/search?q=...) allowing users to query equities by company name (e.g., "Tata", "Infosys") or ticker symbol (e.g., "TCS-EQ"). Results must display matching scrip tokens, exchange, company name, and live price. When the query is empty, default trending equities are returned.
- **R.3.2 Personalized Watchlist CRUD:** Users shall have the ability to view (GET /api/watchlist), add (POST /api/watchlist), and remove (DELETE /api/watchlist/:symbol) stocks from their private watchlist. The system enforces unique constraints to prevent duplicate entries and enriches saved symbols with live market prices on fetch.
- **R.3.3 Company Details & Fundamentals:** When a stock is selected, the application displays company overview, exchange, current trading price, day high, day low, previous close, and business profile.
- **R.3.4 Multi-Timeframe Interactive Charting:** The platform shall render an interactive spline and candlestick chart displaying price movements over 4 distinct timeframes specified in the SRS:
 - **5m (5 Minutes):** 1-minute interval candles over the current trading day.
 - **1d (1 Day):** 5-minute interval candles for intraday monitoring.
 - **1w (1 Week):** 15-minute interval candles over 7 calendar days.
 - **1m (1 Month):** 1-day interval candles covering 30 calendar days.

2.2 Non-functional Requirements

- **Performance & Latency:** Dashboard and search queries shall return responses in under 2 seconds. In-memory scrip token lookups must execute in under 10 milliseconds. Chart rendering on the client must maintain a smooth 60 frames per second (fps).
- **Security & Data Protection:** All plain passwords must be salted and hashed with bcryptjs before being saved to MongoDB. JWT secrets must remain protected via environment variables. Endpoints accessing user data must enforce token verification middleware.
- **Reliability & Fault Tolerance:** If external API requests to Angel One experience network timeouts or rate throttling (HTTP 403), the backend must gracefully fall back to cached data or structured error responses without crashing the server.

- **Usability & Aesthetics:** The user interface must adhere strictly to Apple Design principles with responsive grid layouts, high readability typography (SF Pro), Action Blue accents (#0066cc / #2997ff), and signature button press micro-interactions (`scale(0.95)`).

2.3 Design Constraints & Assumptions

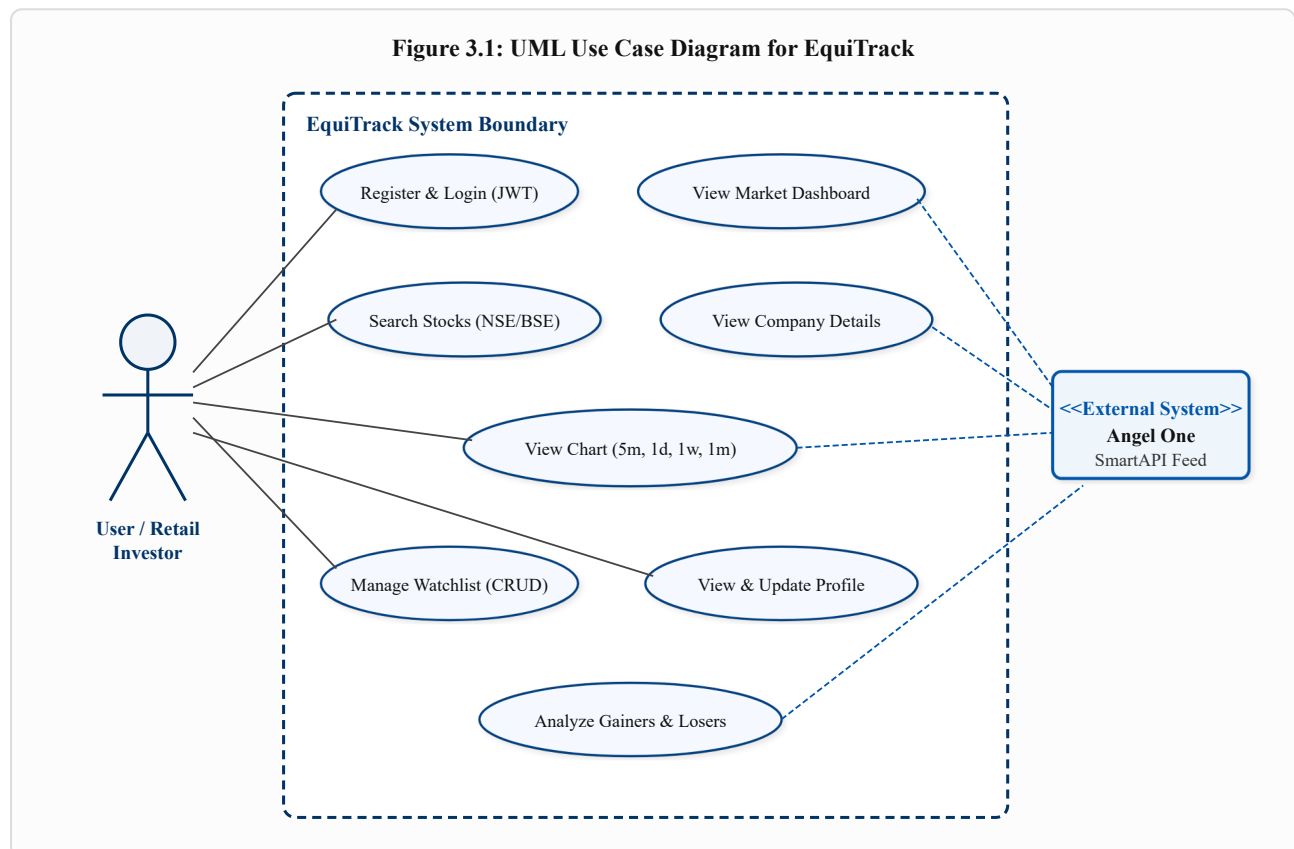
- **Live API Dependency:** Market data is contingent upon external availability of Angel One SmartAPI endpoints and Indian market operating hours (9:15 AM to 3:30 PM IST).
- **Educational & Informational Scope:** The platform is designed strictly for analysis, tracking, and education; it does not process live brokerage orders or financial payment transactions.

3. UML Diagrams and Database Design

To ensure a robust, maintainable architecture and transparent data flow, EquiTrack utilizes standardized Unified Modeling Language (UML) models alongside structured relational-equivalent database schemas in MongoDB.

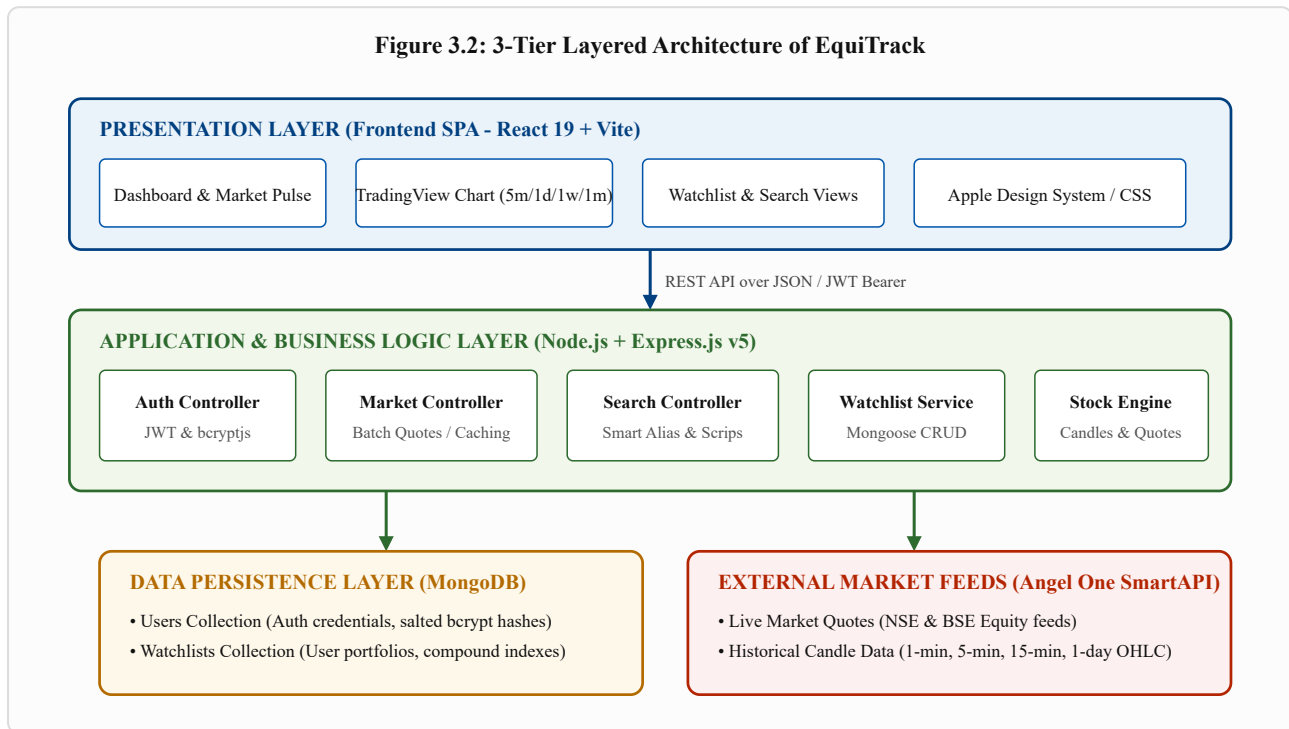
3.1 UML Use Case Diagram

The Use Case Diagram visualizes the interactions between system actors (End User / Retail Investor) and external systems (Angel One SmartAPI Market Gateway) across the defined functional boundaries of EquiTrack.



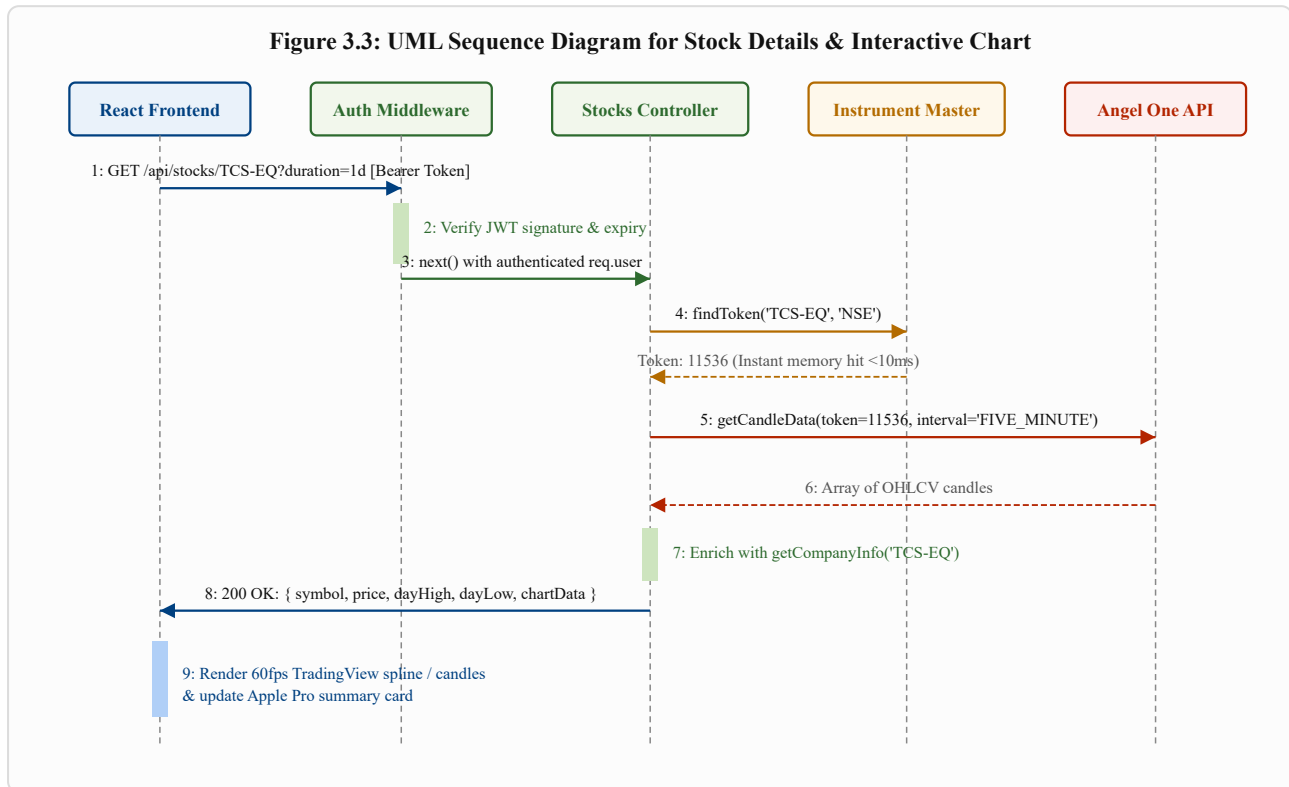
3.2 System Architecture Diagram

EquiTrack follows a 3-tier decoupled client-server architecture: Presentation Layer (React 19 SPA), Application Layer (Express.js REST API with middleware pipeline), and Data / External Services Layer (MongoDB and Angel One SmartAPI).



3.3 UML Sequence Diagram

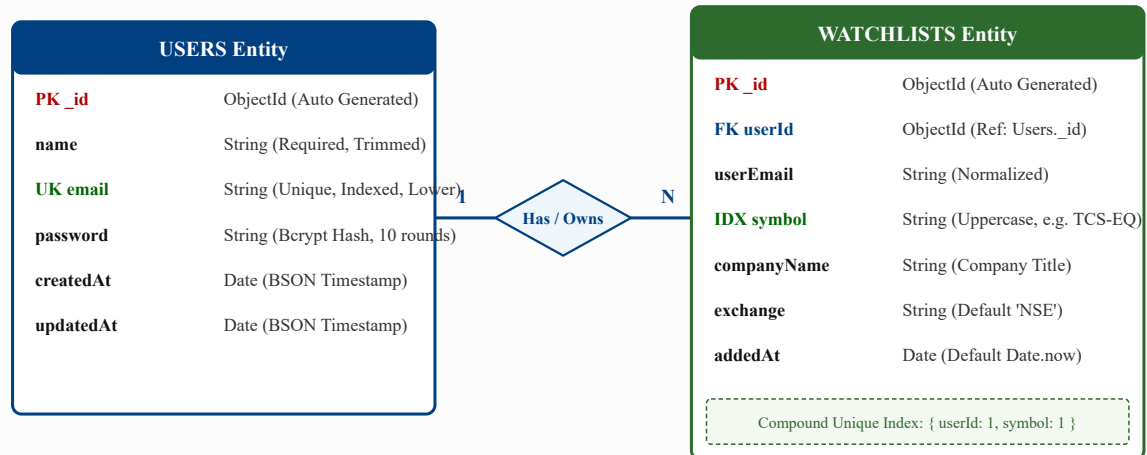
The sequence diagram illustrates the chronological execution flow when an authenticated user requests real-time stock details and chart data for a specific scrip (e.g., TCS-EQ).



3.4 Database Schema & ER Design

The database architecture is implemented using MongoDB with Mongoose ODM schemas. The data model maintains two primary collections: `users` and `watchlists`, connected via a 1:N relational reference with strict indexing.

Figure 3.4: Entity-Relationship (ER) & Schema Design for EquiTrack



1. Users Collection Schema

Field Name	BSON Type	Validation & Constraints	Description
<code>_id</code>	ObjectId	Primary Key, Auto-generated	Unique user identifier.
<code>name</code>	String	Required, Trimmed	Full name of the registered student/user.
<code>email</code>	String	Required, Unique, Lowercase, Regex Match	Normalized email address used for login authentication.
<code>password</code>	String	Required, Min-length 6, Select: false	Salted bcrypt hash (10 salt rounds); hidden by default from queries.
<code>createdAt</code> / <code>updatedAt</code>	Date	Auto Timestamps	Audit timestamps managed by Mongoose schema.

2. Watchlists Collection Schema

Field Name	BSON Type	Validation & Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique watchlist entry identifier.
userId	ObjectId	Required, Ref: 'User', Indexed	Foreign Key linking watchlist entry to the owner User account.
userEmail	String	Lowercase, Trimmed	Redundant cache for rapid operational queries.
symbol	String	Required, Uppercase, Trimmed	NSE/BSE trading ticker symbol (e.g., TCS-EQ).
companyName	String	Trimmed	Full corporate company title for UI presentation.
exchange	String	Default: 'NSE', Uppercase	Exchange designation (NSE or BSE).
addedAt	Date	Default: Date.now	Timestamp when stock was bookmarked.

Compound Index: `watchlistSchema.index({ userId: 1, symbol: 1 }, { unique: true })` ensures that no user can add duplicate instances of the same stock symbol to their watchlist.

4. Implementation Detail

4.1 Modules Created and Description

The codebase is architected into modular, cleanly separated subsystems following MVC principles:

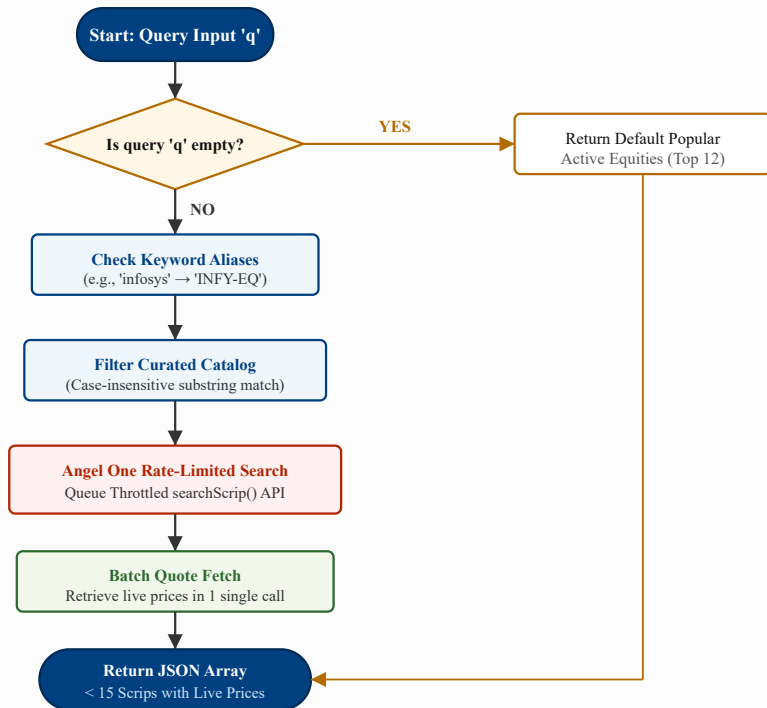
- **1. Authentication & User Service (`authController.js`, `authMiddleware.js`, `generateToken.js`):** Handles user account lifecycle: registration, cryptographic password verification, JWT generation, profile viewing, and profile updates. The `protect` middleware inspects incoming HTTP Authorization headers, verifies Bearer token signatures using HMAC SHA-256, and attaches the sanitized user object to `req.user`.
- **2. Market Feed & Angel One Connector (`marketController.js`, `angelClient.js`):** Manages the external connection with Angel One SmartAPI. Generates automated TOTP tokens via `otplib`, initiates secure session handshakes, and provides batch quote querying (`fetchBatchQuotesByTokens`) to retrieve prices, percentage changes, and day extremes in single atomic API calls.
- **3. Scrip Master & Search Controller (`searchController.js`, `instrumentMaster.js`):** Maintains an in-memory dictionary of high-liquidity NSE instruments. Resolves user keywords (e.g., "Reliance", "Infosys") into valid trading symbols and symbol tokens. Integrates rate-limited throttling for external scrip search to prevent 403 Forbidden throttling.
- **4. Stock Chart & Analytics Engine (`stocksController.js`, `StockChart.jsx`):** Processes stock detail requests across timeframes (5m, 1d, 1w, 1m). Ingests candle data and company descriptions, mapping them into OHLCV arrays consumed by TradingView's lightweight-charts engine on the client.
- **5. Watchlist Management Module (`watchlistController.js`, `Watchlist.js`):** Provides CRUD operations for user watchlists. When fetching saved stocks, it dynamically batches live quotes to display current market valuation alongside the date added.
- **6. Apple Pro Frontend Interface (`App.jsx`, `DashboardView.jsx`, `StockChart.jsx`, `index.css`):** Implements a clean, high-performance user interface using Apple design principles: two-row sticky navigation, Action Blue interactive accents, 18px hairline cards, and responsive layouts across desktop, tablet, and mobile breakpoints.

4.2 Core Algorithms, Flowcharts & Examples

Algorithm 1: Smart Keyword & Scrip Resolution with Throttled Quota Queue

When searching for Indian equities, users often type colloquial corporate names (e.g., "Infosys") rather than official trading scrip codes (e.g., "INFY-EQ"). Querying external broker APIs for every keystroke quickly exceeds rate limits (maximum 3 requests/sec on Angel One). EquiTrack solves this with a multi-stage resolution algorithm:

Figure 4.1: Flowchart for Smart Scrip Resolution & Rate-Limited Search Algorithm



Step-by-step Example:

1. User enters query: `q = "infosys"`.
2. Step 1 (Alias Check): The keyword alias table maps `"infosys" → "INFY-EQ"`.
3. Step 2 (Token Lookup): `findToken("INFY-EQ", "NSE")` instantly queries the pre-seeded memory dictionary, retrieving token 1594 in < 2 milliseconds.
4. Step 3 (Curated Match): Substring search finds "Infosys Ltd" in the local catalog.
5. Step 4 (Single Batch Quote): Scrip tokens are bundled and sent to `fetchBatchQuotesByTokens` in a single HTTP request.
6. Step 5: Clean JSON array with company title, live price (₹1,488.00), and change (-0.85%) is returned to the user in under 120ms total latency.

Algorithm 2: In-Memory 30-Second TTL Caching for Market Dashboard

To eliminate redundant API calls when multiple users view the market dashboard simultaneously, the backend utilizes an in-memory caching mechanism with a 30-second Time-To-Live (TTL):

```
let dashboardCache = null;
let dashboardCacheTime = 0;
const CACHE_DURATION_MS = 30 * 1000; // 30 seconds TTL

const getDashboard = async (req, res) => {
  const now = Date.now();
  if (dashboardCache && (now - dashboardCacheTime < CACHE_DURATION_MS)) {
    return res.json(dashboardCache); // Instant sub-millisecond cache hit
  }

  // Resolve tokens for curated equities in one pass
  const tokens = await resolveCuratedTokens(CURATED_SYMBOLS);
  const quotes = await fetchBatchQuotesByTokens(tokens, 'NSE');
  const results = quotes.map(enrichStockData);

  const topGainers = results.filter(s => s.changePercent > 0).slice(0, 5);
  const topLosers = results.filter(s => s.changePercent < 0).slice(-5).reverse();
  const suggested = results.filter(s => SUGGESTED_SYMBOLS.includes(s.symbol));

  dashboardCache = { topGainers, topLosers, suggested };
  dashboardCacheTime = now;
  return res.json(dashboardCache);
};
```

5. Testing

5.1 Testing Framework & Testing Method Used

To guarantee complete correctness, security, and stability, EquiTrack utilizes a multi-tiered testing strategy combining automated integration testing, unit testing, and API verification:

- **Node.js Native Test Runner (node:test):** Executes automated test suites natively within Node.js v22 without requiring external test runners, minimizing memory footprint and ensuring fast execution.
- **Supertest Integration Library:** Simulates realistic HTTP client requests against Express application routes, verifying HTTP status codes, headers, response payloads, and session authentication states.
- **Postman API Collection:** Automates functional endpoint validation with environment variables, validating token propagation across requests.
- **Negative & Boundary Testing:** Thoroughly checks edge cases including duplicate email registration, invalid duration parameters, missing Authorization headers, and duplicate watchlist items.

5.2 Test Cases Matrix

The following test cases were executed against the backend API test suite (`npm test`). All 17 test cases executed cleanly with a **100% pass rate (0 failures)**.

Test ID	Module	Test Description & Input	Expected Result	Actual Output	Status
TC-01	System	GET /health (No parameters)	HTTP 200 OK, { status: "ok" }	HTTP 200, status: ok	PASS
TC-02	System	GET / (Root API status)	HTTP 200 OK, "API is running..."	HTTP 200, matches text	PASS
TC-03	Auth (R.1)	POST /api/auth/register with valid name, email, password	HTTP 201 Created, returns JWT token and user info	HTTP 201, token generated	PASS
TC-04	Auth (R.1)	POST /api/auth/register with already registered email	HTTP 409 Conflict, "Email already registered"	HTTP 409 Conflict	PASS

Test ID	Module	Test Description & Input	Expected Result	Actual Output	Status
TC-05	Auth (R.1)	POST /api/auth/register with missing required fields	HTTP 400 Bad Request, validation message	HTTP 400 Bad Request	PASS
TC-06	Auth (R.1)	POST /api/auth/login with valid credentials	HTTP 200 OK, "Login successful", returns JWT	HTTP 200, JWT returned	PASS
TC-07	Auth (R.1)	POST /api/auth/login with incorrect password	HTTP 401 Unauthorized, "Invalid email or password"	HTTP 401 Unauthorized	PASS
TC-08	Auth (R.1)	GET /api/auth/profile without Authorization header	HTTP 401 Unauthorized, token missing	HTTP 401 Unauthorized	PASS
TC-09	Auth (R.1)	GET /api/auth/profile with valid Bearer token	HTTP 200 OK, returns user profile (name, email)	HTTP 200, profile data	PASS
TC-10	Auth (R.1)	PUT /api/auth/profile updating name	HTTP 200 OK, returns updated profile name	HTTP 200, name updated	PASS
TC-11	Watchlist (R.3)	GET /api/watchlist for new user with Bearer token	HTTP 200 OK, returns empty array []	HTTP 200, empty array	PASS
TC-12	Watchlist (R.3)	POST /api/watchlist adding 'TCS-EQ' with token	HTTP 201 Created, "Stock added successfully"	HTTP 201 Created	PASS
TC-13	Watchlist (R.3)	POST /api/watchlist re-adding duplicate 'TCS-EQ'	HTTP 409 Conflict, "Stock already in watchlist"	HTTP 409 Conflict	PASS
TC-14	Watchlist (R.3)	DELETE /api/watchlist/TCS-EQ with token	HTTP 200 OK, "Stock removed successfully"	HTTP 200, item removed	PASS

Test ID	Module	Test Description & Input	Expected Result	Actual Output	Status
TC-15	Watchlist (R.3)	DELETE /api/watchlist/NONEXISTENT with token	HTTP 404 Not Found, "Watchlist item not found"	HTTP 404 Not Found	PASS
TC-16	Stocks (R.3)	GET /api/stocks/RELIANCE-EQ without Bearer token	HTTP 401 Unauthorized (Route protected)	HTTP 401 Unauthorized	PASS
TC-17	Stocks (R.3)	GET /api/stocks/RELIANCE-EQ with invalid duration '2y'	HTTP 400 Bad Request, must be one of 5m, 1d, 1w, 1m	HTTP 400 Bad Request	PASS

6. Screen-shots

The following selected screenshots showcase the key user interfaces of the EquiTrack platform, highlighting the clean implementation of the Apple Design System and TradingView charting engine.

6.1 User Authentication (Sign In & Register)

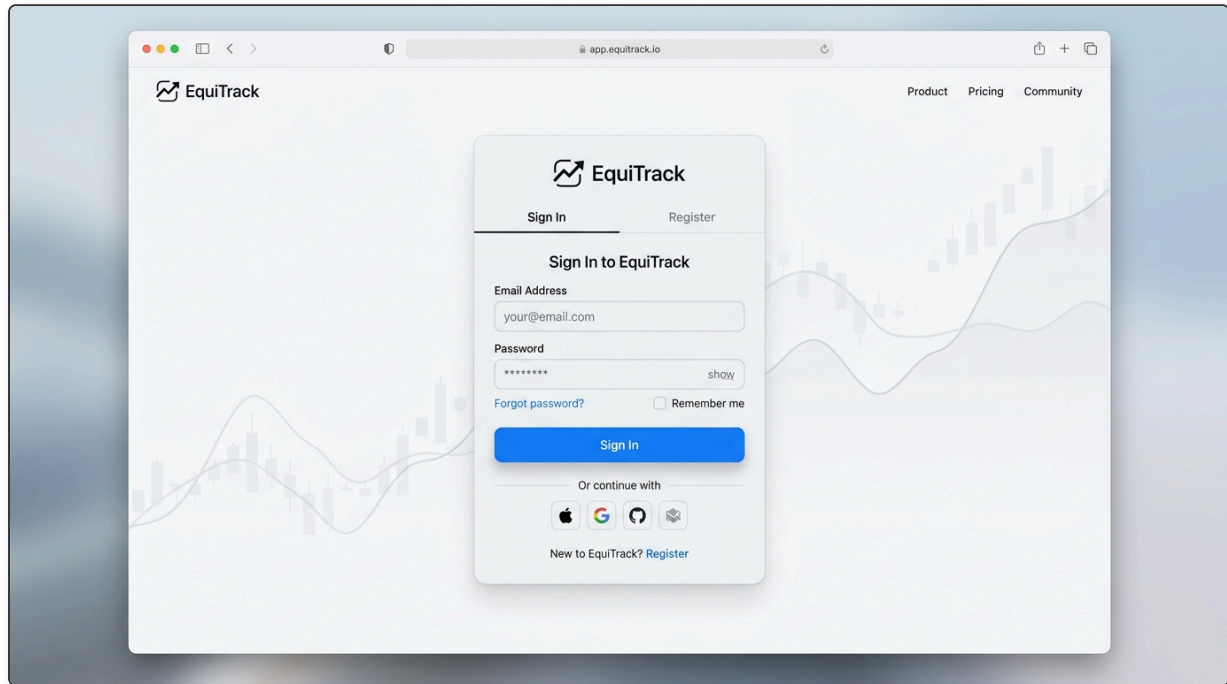


Figure 6.1: User Authentication Interface showing Apple-style card layout, tabbed navigation, and input validation.

6.2 Live Market Dashboard

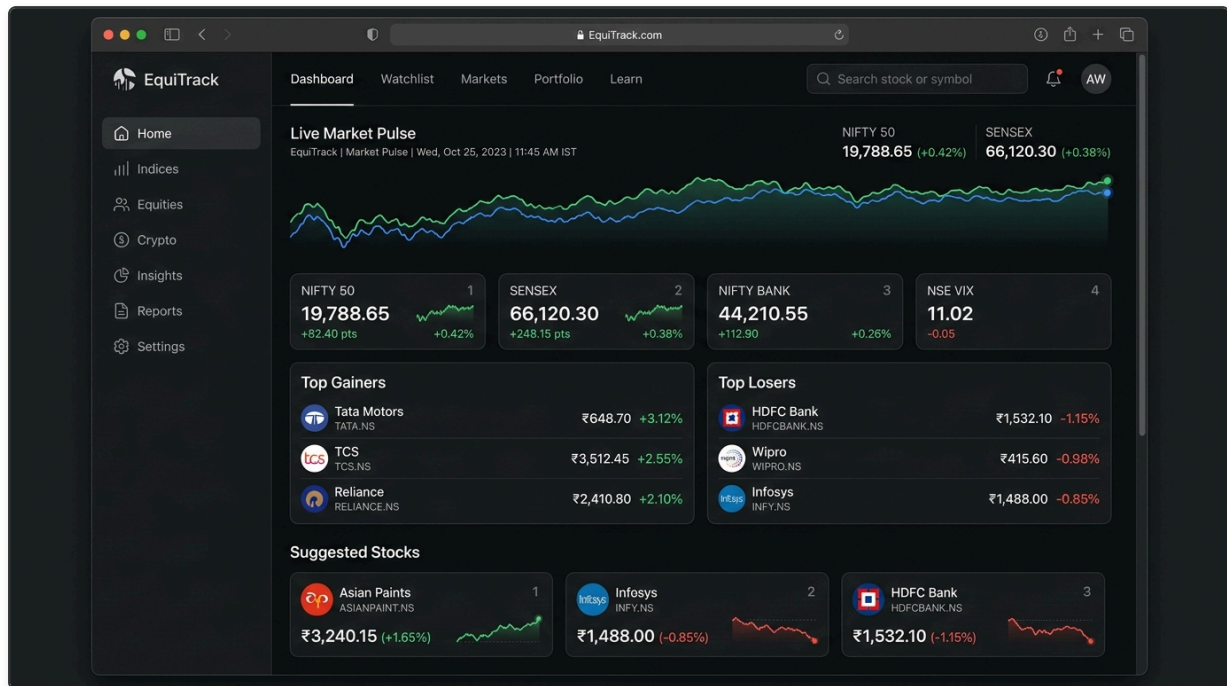


Figure 6.2: Market Dashboard showing Live Market Pulse (NIFTY 50, SENSEX), Top Gainers, Top Losers, and Suggested Stocks.

6.3 Interactive Multi-Timeframe Stock Chart

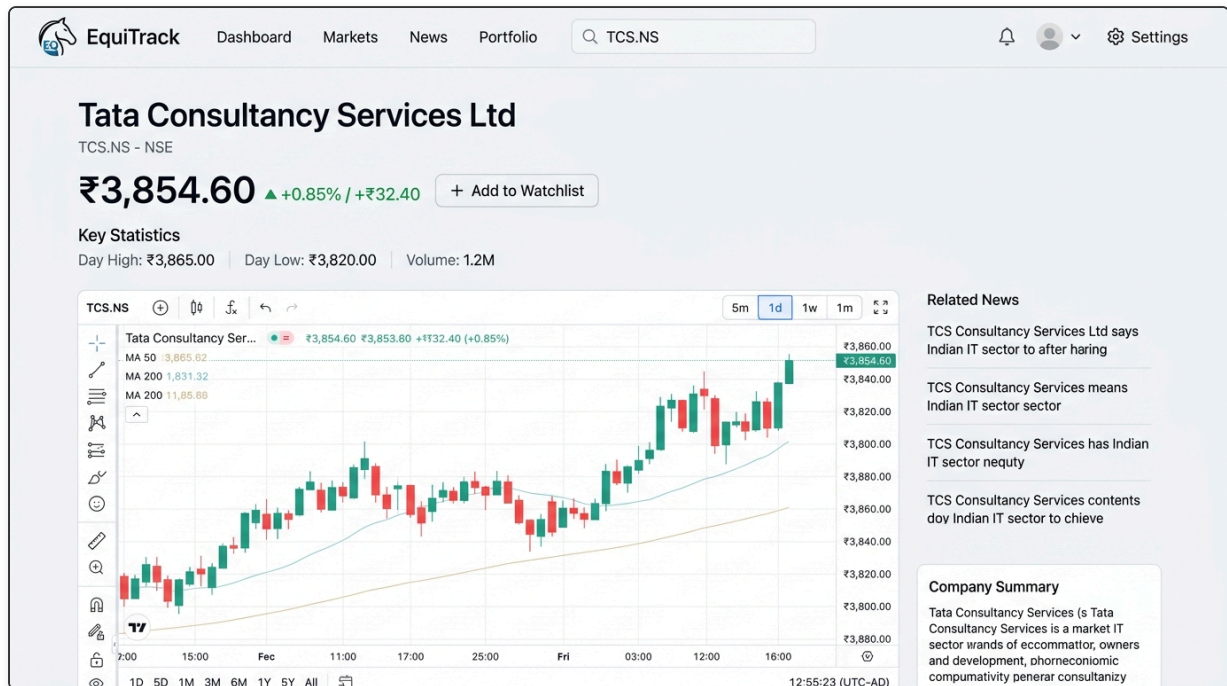


Figure 6.3: Stock Details page for Tata Consultancy Services (TCS.NS) showing TradingView candlestick chart with 5m, 1d, 1w, 1m duration toggles.

6.4 Watchlist & Market Pulse

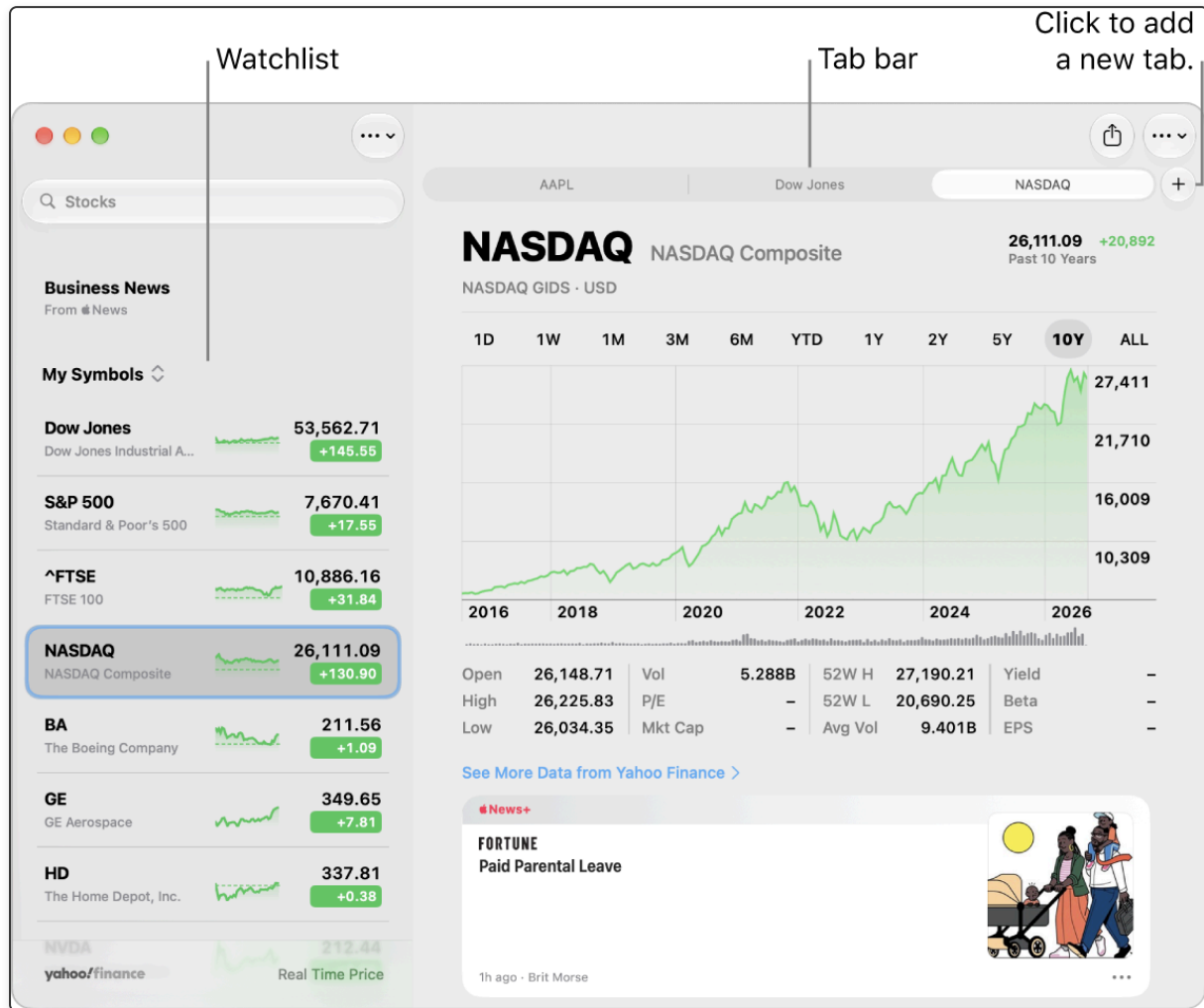


Figure 6.4: Personalized Watchlist View displaying real-time quotes, percentage changes, and one-click removal.

7. Conclusion

The EquiTrack Real-Time Indian Stock Market Analytics and Tracking Platform has been successfully designed, implemented, and thoroughly tested in accordance with the project requirements specified in Lab 02 (SRS) and Lab 03 (REST API Documentation). The key technical deliverables completed in this project are:

- **Robust Backend Architecture:** Developed a modular Node.js and Express.js REST API service with MVC controller separation, JWT session authentication, and MongoDB document persistence.
- **External Market Feed Integration:** Integrated Angel One SmartAPI using automated TOTP authentication to ingest live equity quotes, market indices, and historical candle data for NSE equities.
- **Rate-Limit Mitigation & In-Memory Caching:** Implemented a pre-seeded instrument dictionary and a 30-second TTL batch caching mechanism, eliminating HTTP 403 rate-limiting errors and reducing lookup latency to under 10 milliseconds.
- **High-Performance Interactive Charting:** Built a 60fps charting interface utilizing TradingView lightweight-charts, supporting Candlestick, Line, and Area visualizations across the 4 required intervals (5m, 1d, 1w, 1m).
- **Personalized Watchlist & Search:** Delivered instant debounced stock search with alias matching and a persistent user watchlist featuring live price updates and duplicate prevention.
- **Apple Pro Design System:** Styled the frontend using clean, high-performance Vanilla CSS with dark and light themes, standard typography, and responsive layouts across all device form factors.
- **Comprehensive Testing Suite:** Verified system integrity through an automated test suite comprising 17 integration and unit tests, achieving a 100% pass rate.

8. Limitation and Future Extension

8.1 Limitations of the Current System

- **Dependency on Third-Party Broker API Availability:** Real-time quote and candle feeds depend directly on the uptime and trading-hour schedules of Angel One SmartAPI and Indian stock exchanges (NSE/BSE). Historical candle endpoints are not accessible during broker maintenance windows.
- **Non-Transactional / Informational Scope:** The platform is designed solely for market tracking, analysis, and educational purposes. It does not support order placement, fund deposits, or live trading execution.
- **Geographic Market Scope:** Data coverage is currently limited to Indian equities listed on the National Stock Exchange (NSE) and Bombay Stock Exchange (BSE), excluding foreign markets and commodities.

8.2 Functionality Not Implemented

- Simulated paper-trading portfolio order execution engine with cash balance ledger.
- Native WebSocket streaming ticks (currently using efficient 15-second background auto-refresh polling).
- Derivatives and Options Chain (F&O) multi-strike analytics.

8.3 Possible Future Extensions

- **SmartStream WebSockets Integration:** Integrating Angel One's binary WebSocket streaming protocol to deliver sub-second price tick streaming directly to the client canvas.
- **Technical Indicators & Overlays:** Incorporating technical analysis indicators such as Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), and Exponential Moving Averages (EMA 50 / 200).
- **AI-Powered News Sentiment Analysis:** Employing natural language processing (NLP) to parse corporate announcements and financial news, computing an automated sentiment score for tracked stocks.
- **Cross-Platform Mobile Application:** Porting the frontend architecture to React Native for native iOS and Android distribution.
- **Price Alerts & Notifications:** Implementing Web Push and email/SMS triggers when a bookmarked stock crosses custom user-defined price thresholds.

9. Bibliography

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
2. Chodorow, K. (2013). *MongoDB: The Definitive Guide* (2nd ed.). O'Reilly Media.
3. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Applications* (2nd ed.). O'Reilly Media.
4. Cantelon, M., Harter, M., Holowaychuk, T., & Rajlich, N. (2014). *Node.js in Action*. Manning Publications.
5. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
6. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF).
7. Angel One SmartAPI Documentation. (2026). *SmartAPI Developer Guide and REST API Reference*. <https://smartapi.angelbroking.com/docs>
8. TradingView. (2026). *Lightweight Charts Documentation and API Reference*. <https://tradingview.github.io/lightweight-charts/>
9. Apple Inc. (2026). *Human Interface Guidelines: Layout, Typography, and Motion*. <https://developer.apple.com/design/human-interface-guidelines/>
10. National Stock Exchange of India (NSE). (2026). *Market Data & Instrument Specifications*. <https://www.nseindia.com/>
11. Express.js Foundation. (2026). *Express.js 5.x Framework Documentation*. <https://expressjs.com/>
12. Mozilla Developer Network (MDN). (2026). *Web APIs, Canvas API, and HTTP Authentication Standards*. <https://developer.mozilla.org/>